
COMS W4167: Computer Animation
Programming Assignment 6
Due 9:00PM Monday, May 11th, 2026 (EST)

Academic Honesty Policy

You are permitted and encouraged to discuss your work with other students. Except where explicitly stated otherwise, you may work out equations in writing on paper or a whiteboard. You are encouraged to use **Ed Discussion** to converse with other students, the TAs, and the instructor.

HOWEVER, you must NOT share source code or hard copies of source code. Refrain from activities or the sharing materials that could cause your source code to **APPEAR TO BE** similar to another student's enrolled in this or previous years. We will be monitoring source code for individuality. Cheating will be dealt with severely. Source code should be yours and yours only. Do not cheat. For more details, please refer to the full academic honesty policy on the Engineering School's website.

Chapter 1

Course Notes: Fluid Simulation

1.1 Staggered Grid (MAC Grid)

There are more than one style of grid we can use in an Eulerian simulation. In the seminar Stable Fluids paper [2], the grid is called a *collocated grid*, which means all the quantities such as velocities, pressure and dye density are defined at grid cell centers. This type of grid gives an intuitive velocity vector defined at cell centers, which points towards the fluid flow direction, and interpolations can be done easily. However when it comes to enforcing incompressibility, there is a serious disadvantage to it: discretizing the differential operator through finite differencing is a bit tricky. Central differencing gives a bad formula that only depends on neighboring values and not the value we are looking at, thus incorrectly returning zero for a sawtooth shaped field; forward/backward differencing is biased. See [1], section 2.4 *Grids*.

This motivates the *staggered grid*, first introduced in a 1965 paper by Harlow and Welch under the name *MAC grid* (Marker-And-Cell), which is the approach we'll be using. The idea is to define pressure and dye density at cell centers, and velocities on cell walls. In our 2D case, it means the horizontal velocities are defined on vertical walls, and vice versa (shown in figure 1.1). With this grid, the differential operator is easily discretized:

$$\frac{\partial u_{i,j}}{\partial x} = \frac{u_{i,j+\frac{1}{2}} - u_{i,j-\frac{1}{2}}}{h} \quad (1.1)$$

where h is the width (and height) of a grid cell. Can you see why this is more elegant than what the collocated grid will give you? Here's a list of all the finite differencing discretizations we may need (assume the positive direction of u , x and j is pointing to the right, and positive direction of v , y and i is pointing down):

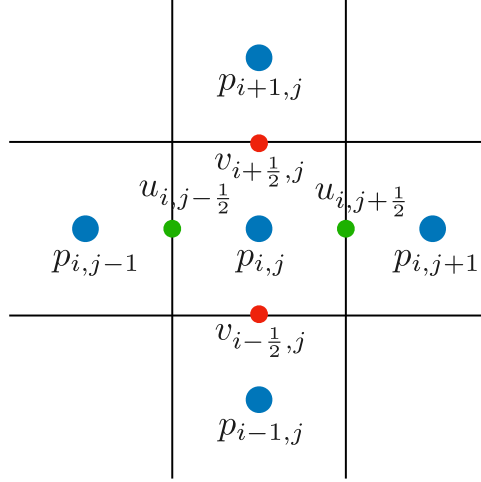


Figure 1.1: A staggered grid.

$$\begin{aligned}
\nabla \cdot \mathbf{u}_{i,j} &= \frac{\partial u_{i,j}}{\partial x} + \frac{\partial v_{i,j}}{\partial y} \\
\frac{\partial u_{i,j}}{\partial x} &= \frac{u_{i,j+\frac{1}{2}} - u_{i,j-\frac{1}{2}}}{h} \\
\frac{\partial v_{i,j}}{\partial y} &= \frac{v_{i+\frac{1}{2},j} - v_{i-\frac{1}{2},j}}{h} \\
\Delta p_{i,j} &= \frac{p_{i+1,j} + p_{i-1,j} + p_{i,j+1} + p_{i,j-1} - 4p_{i,j}}{h^2} \\
\frac{\partial p_{i,j+\frac{1}{2}}}{\partial x} &= \frac{p_{i,j+1} - p_{i,j}}{h} \\
\frac{\partial p_{i+\frac{1}{2},j}}{\partial y} &= \frac{p_{i+1,j} - p_{i,j}}{h}
\end{aligned}$$

The bold face $\mathbf{u}$ denotes the velocity vector (u, v) , with the horizontal component being u and vertical component being v . Make sure you understand these formula well by looking at figure 1.1 and/or drawing your own diagrams. You'll need it when implementing this PA.

One of the neat things about the staggered grid is, you can think of the velocities as flows through the walls it is defined on. Since the velocity is always perpendicular to the wall and defined the at the center of the wall, it naturally represents how much the fluid is flowing through that wall, into or out of the cell on either side. This view can be helpful when trying to understand incompressibility.

However if we want to convert the formula above to Python code, a problem immediately shows up: we can't use fractions like $j + \frac{1}{2}$ as an array index. Of course we can choose to store it in the j -th element in the array, or the $(j+1)$ -th element: it's pretty much an arbitrary decision. In the actual implementation, we have to define some convention for ourselves to follow, to map fractional indices to integer indices.

We follow the convention outlined in figure 1.2. The blue dots are where we define pressure, having size $N \times M$. The green dots and red dots are horizontal (u) and vertical (v) velocities respectively, and have size $N \times (M + 1)$ and $(N + 1) \times M$, respectively. The indexing all starts from 0. As it can be seen clearly from

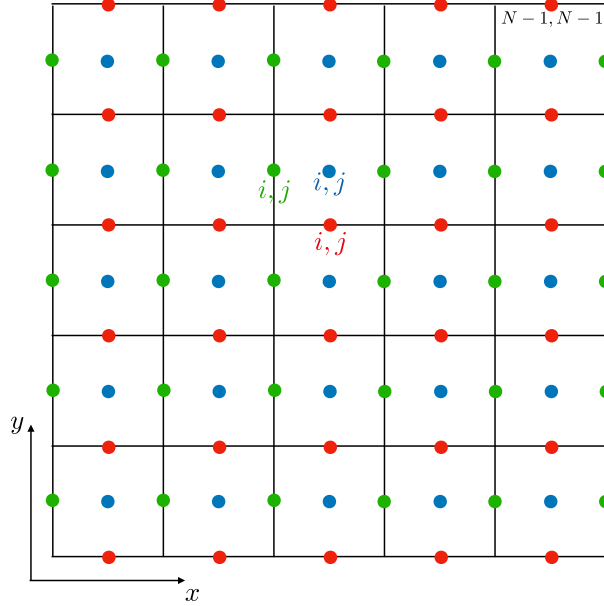


Figure 1.2: The indexing convention.

the diagram, $u_{i,j-\frac{1}{2}}$ has index (i, j) , and $v_{i-\frac{1}{2},j}$ has index (i, j) too, so equation 1.1 can be implemented as:

$$\frac{\partial u_{i,j}}{\partial x} = \frac{u(i, j+1) - u(i, j)}{h}.$$

Although the staggered grid shines at finite differencing, it does encounter some trouble when we want to sample the velocity at an arbitrary location. As explained in the Stable Fluids paper, this ability to sample velocity is needed the semi-Lagrangian method for advection: we trace a imaginary particle backward in time along the velocity field to its position at the previous time step, and read off quantities (density or velocity) there in order to fill them in the particle's current location, which requires interpolation of quantities at nearby grid points. Interpolation with staggered grid is a little bit trickier since grid points for different quantities are located at different positions in space, requiring a different interpolation routine for each quantity. The good news is, if we only look at dots of a single color and ignore other dots in figure 1.2, it is obvious that these dots still form a regular square grid, which means the bilinear interpolation still works, and we just need to offset the positions accordingly. Figure 1.3 demonstrates this idea. If you are interested in vertical velocities (defined at red dots) at the location of the yellow dot, you'll bilinearly interpolate the four red dots as shown on the right. Other quantities are similarly interpolated.

Note: Although you will be implmenting the FLIP method [3] in this PA, I strongly suggest you read the Stable Fluids paper [2] first, and understand the semi-Lagrangian method for advection and pressure projection for incompressibility enforcement, as it is the foundation of the FLIP method.

Implementation Tip: The starter code follows the indexing convention outlined in figure 1.2. In the `nemo.sim.State` class, the field `fluid_u_x` and `fluid_u_y` store the horizontal and vertical velocities, respectively.

1.2 Pressure Projection

The pressure projection step happens after the advection step in the Stable Fluids paper. It is used to enforce incompressibility by projecting the velocity field onto a divergence-free space. The pressure projection step

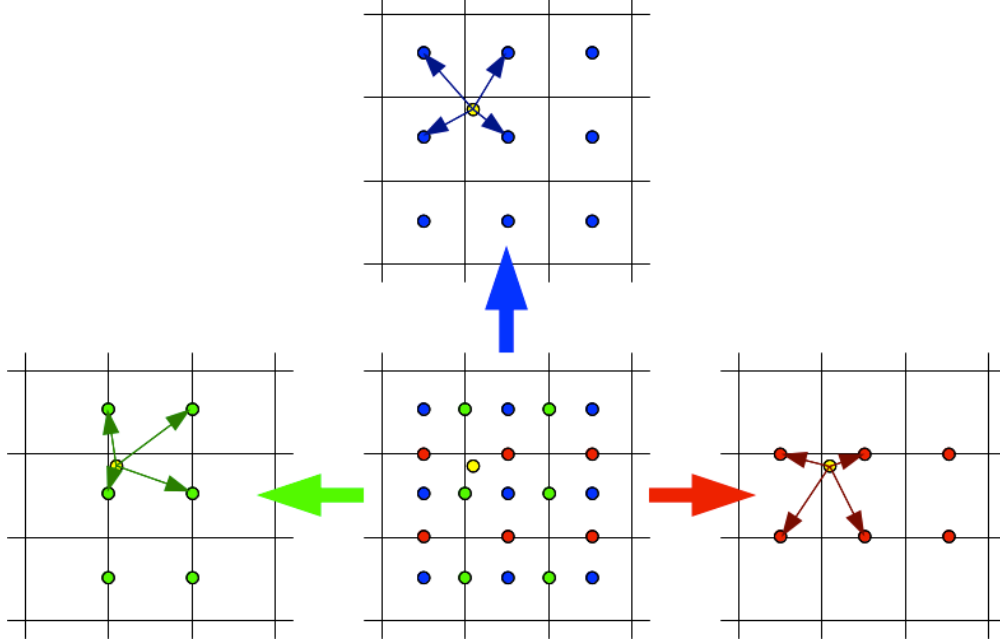


Figure 1.3: Interpolation for different quantities. We'd like to interpolate the red (v), green (u) and blue (density) values at the yellow dot's location. The grid nodes to use for interpolating each quantity is shown in a separate diagram.

is given by the following equation:

$$\frac{\Delta t}{\rho} \Delta p = \nabla \cdot \mathbf{u}^*, \quad (1.2)$$

where $\mathbf{u}^*$ is the velocity field after advection. This is a Poisson equation, which involves only the spatial derivatives of pressure (no time derivatives). Therefore, we can discretize it using the finite difference method on the staggered grid. For a given cell (i, j), the discretized equation is given by:

$$\frac{\Delta t}{\rho} \left(\frac{4p_{i,j} - p_{i+1,j} - p_{i-1,j} - p_{i,j+1} - p_{i,j-1}}{h^2} \right) = - \left(\frac{u_{i+\frac{1}{2},j}^* - u_{i-\frac{1}{2},j}^*}{h} + \frac{v_{i,j+\frac{1}{2}}^* - v_{i,j-\frac{1}{2}}^*}{h} \right), \quad (1.3)$$

where u^* and v^* are the horizontal and vertical velocities stored on the staggered grid after advection, respectively.

If there are N water cells, this discretization leads to a linear system of size $N \times N$. This linear system is sparse and symmetric (**Implementation Tip:** when debugging your code, you could sanity check the symmetry of the linear system you assembled). In practice, we can solve this linear system using the preconditioned conjugate gradient method. The conjugate gradient method is an iterative method for solving linear systems of the form $Ax = b$. In the starter code, we illustrated the use of `scipy.sparse.linalg.cg` to solve this linear system.

After solving the linear system, we got the pressure value $p_{i,j}$ for all water cells. We can then update the velocity field using

$$\begin{aligned} u_{i,j-\frac{1}{2}} &= u_{i,j-\frac{1}{2}}^* - \frac{\Delta t}{\rho} \frac{p_{i,j} - p_{i,j-1}}{h} \\ v_{i-\frac{1}{2},j} &= v_{i-\frac{1}{2},j}^* - \frac{\Delta t}{\rho} \frac{p_{i,j} - p_{i-1,j}}{h}. \end{aligned} \quad (1.4)$$

1.3 Boundary Conditions

When simulating fluids one must pay careful attention to boundary conditions. There are natural boundaries to the liquids we see every day, such as a solid container which doesn't allow the liquid to pass and restricts its shape (solid boundary condition). There are also boundaries between water and air (free boundary condition). If a fluid grid cell is at the boundary, recall that the new velocities on the boundary faces involve pressures outside the fluid that we have to define through boundary conditions. For example, if grid cell $(i, j + 1)$ is an air cell, then we replace $p_{i,j+1}$ in Eq. (1.7) with zero. This is because the water density is 1000 times larger than the air density, so the pressure difference is dominated by the water pressure, and it is reasonable to assume the air pressure is zero. If grid cell $(i + 1, j)$ is a solid cell, then we replace $p_{i+1,j}$ with the value we compute from the boundary condition there, as in formula (1.4). Assuming $(i - 1, j)$ and $(i, j - 1)$ are fluid cells, this would reduce the equation to the following:

$$\frac{\Delta t}{\rho} \left(\frac{4p_{i,j} - \left[p_{i,j} + \frac{\rho h}{\Delta t} (u_{i+\frac{1}{2},j}^* - u_{\text{solid}}) \right] - p_{i-1,j} - 0 - p_{i,j-1}}{h^2} \right) = - \left(\frac{u_{i+\frac{1}{2},j}^* - u_{i-\frac{1}{2},j}^*}{h} + \frac{v_{i,j+\frac{1}{2}}^* - v_{i,j-\frac{1}{2}}^*}{h} \right), \quad (1.5)$$

where u_{solid} is the velocity of the solid boundary—in this PA, the solid boundary is always at rest, so you can set u_{solid} to 0. The above equation can be rewritten to move the constant term to the right side:

$$\frac{\Delta t}{\rho} \left(\frac{3p_{i,j} - p_{i-1,j} - p_{i,j-1}}{h^2} \right) = - \left(\frac{u_{i+\frac{1}{2},j}^* - u_{i-\frac{1}{2},j}^*}{h} + \frac{v_{i,j+\frac{1}{2}}^* - v_{i,j-\frac{1}{2}}^*}{h} \right) + \left(\frac{u_{i+\frac{1}{2},j}^* - u_{\text{solid}}}{h} \right). \quad (1.6)$$

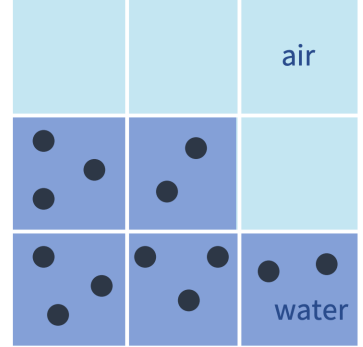
Implementation Tip: You need to carefully set up the linear system for pressure projection. Equation (1.6) only illustrates the case where $(i, j + 1)$ is an air cell and $(i + 1, j)$ is a solid cell. For other cases, different terms will be added to the right-hand side. Make sure you understand the derivation of (1.6) to derive correct terms for other cases (they are all very similar, make sure you get the signs right).

In your implementation, you could merge $\frac{\Delta t}{\rho h}$ into the pressure field, and solve for $\tilde{p}_{i,j} = \frac{\Delta t}{\rho h} p_{i,j}$ instead, in this way, you assemble the linear system:

$$4\tilde{p}_{i,j} - \tilde{p}_{i+1,j} - \tilde{p}_{i-1,j} - \tilde{p}_{i,j+1} - \tilde{p}_{i,j-1} = - \left(u_{i+\frac{1}{2},j}^* - u_{i-\frac{1}{2},j}^* + v_{i,j+\frac{1}{2}}^* - v_{i,j-\frac{1}{2}}^* \right), \quad (1.7)$$

and the velocity update (1.4) becomes:

$$\begin{aligned} u_{i,j-\frac{1}{2}} &= u_{i,j-\frac{1}{2}}^* - (\tilde{p}_{i,j} - \tilde{p}_{i,j-1}) \\ v_{i-\frac{1}{2},j} &= v_{i-\frac{1}{2},j}^* - (\tilde{p}_{i,j} - \tilde{p}_{i-1,j}). \end{aligned} \quad (1.8)$$



1.4 PIC/FLIP Method

Let's start with the Particle-in-cell (PIC) method. Its only difference from the Stable Fluids method is the advection step. Instead of using semi-Lagrangian advection (as described in [2]), the PIC method uses particles and advects them. The particles also naturally label the water cells (see Figure 1.4). In general, it has the following steps:

1. Advect particles using the particle velocity (Particles move along their own velocities by Δt).
2. Transfer particle velocity to the staggered grid.
3. Apply gravity to the grid velocity (vertical component only).
4. Solve the pressure on the grid (Poisson equation solve, see [2]).

Figure 1.4: PIC/FLIP method.

5. Update the velocity field using the pressure (1.4).
6. Transfer the velocity field from the grid back to the particles.

Particle to Grid Velocity Transfer. To transfer particle velocity to the staggered grid, we need to map the particle position to the grid index, and accumulate the particle velocity to the grid nodes that surround the particle. The velocity transfer back and forth between the particles and the grid uses bilinear interpolation.

Use Figure 1.5 as an example. Given a particle (in orange in Figure 1.5) position (x_p, y_p) , it locates in a grid cell with index $(\lfloor x_p/h \rfloor, \lfloor y_p/h \rfloor)$, where h is the grid cell size, and its *local* position within that grid cell is (w_x, w_y) , where

$$\begin{aligned} w_x &= \frac{x_p}{h} - \lfloor \frac{x_p}{h} \rfloor \\ w_y &= \frac{y_p}{h} - \lfloor \frac{y_p}{h} \rfloor. \end{aligned} \quad (1.9)$$

When transferring its velocity to the grid, the horizontal velocity transfer considers the four green nodes in Figure 1.5, and the vertical velocity transfer considers the four red nodes. For the particular case shown in Figure 1.5, the local position of the particle with respect to the green nodes is $(w_x^u, w_y^u) = (w_x, w_y - 0.5)$. and the local position of the particle with respect to the red nodes is $(w_x^v, w_y^v) = (w_x - 0.5, w_y)$. Then, the bilinear velocity contributions to the four green nodes are:

$$\begin{aligned} \text{bottom left} &: (1 - w_x^u)(1 - w_y^u)u_p \\ \text{bottom right} &: w_x^u(1 - w_y^u)u_p \\ \text{up left} &: (1 - w_x^u)w_y^u u_p \\ \text{up right} &: w_x^u w_y^u u_p \end{aligned} \quad (1.10)$$

Similar bilinear contributions are computed for the four red nodes. The final velocity of each grid node accumulates contributions from all its surrounding particles, and the total accumulated velocity is then normalized by the total accumulated weights.

Grid to Particle Velocity Transfer. The grid-to-particle velocity transfer is the reverse of the particle-to-grid velocity transfer. Still using Figure 1.5 as an example. If the horizontal velocity stored on the four green nodes are u_0 (bottom-left), u_1 (bottom-right), u_2 (up-left), u_3 (up-right), then the particle velocity is

$$u_p = u_0(1 - w_x^u)(1 - w_y^u) + u_1 w_x^u(1 - w_y^u) + u_2(1 - w_x^u)w_y^u + u_3 w_x^u w_y^u. \quad (1.11)$$

Note that unlike particle-to-grid transfer, here we don't need to normalize u_p by the total weights, because the total weight is always 1.

FLIP Method. The problem with the PIC method is that the velocity transfer back and forth between the particles and the grid smooths out the velocity field. To address this, the FLIP method [3] introduces a blending factor α to blend two ways of computing the particle velocity: 1) for each particle p , it computes a velocity $\mathbf{u}_p^{\text{PIC}}$ using (1.11). 2) At the same time, it computes another velocity $\mathbf{u}_p^{\text{FLIP}}$ by interpolating the *velocity changes*:

$$\mathbf{u}_p^{\text{FLIP}} = \mathbf{u}_p^* + \Delta \mathbf{u}_p, \quad (1.12)$$

where $\mathbf{u}_p^*$ is the particle's current velocity, and $\Delta \mathbf{u}_p$ is the bilinear interpolation of *velocity changes* of the grid velocity fields before and after the pressure projection step, i.e., let the grid velocity field after Step 2 above be $\hat{\mathbf{u}}$, and let the grid velocity field after Step 5 above be $\mathbf{u}$. $\Delta \mathbf{u}_p$ is the bilinear interpolation (as in (1.11)) of $\mathbf{u} - \hat{\mathbf{u}}$ at the particle's location.

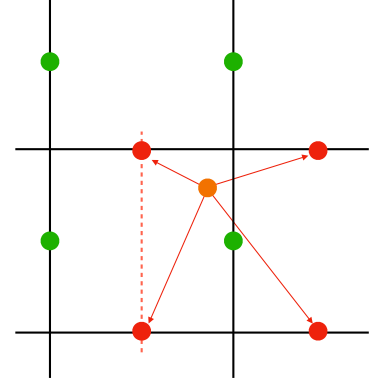


Figure 1.5: Velocity transfer between particles and grid.

Finally, the particle velocity is blended using the blending factor α :

$$\mathbf{u}_p = (1 - \alpha)\mathbf{u}_p^{\text{PIC}} + \alpha\mathbf{u}_p^{\text{FLIP}}. \quad (1.13)$$

In practice, we can set α to 0.95, and stored in `FluidFlipSolver.FLIP_FACTOR`.

Please refer to Sec. 4 of paper [3] for more details.

Implementation Tips: 1) Since the pressure projection solve only considers the water cells and the water cells change over time, the size of the pressure projection linear system also varies over time. You need to label each cell's type (water or air) after the particle advection step, and create a map that maps each water cell to its index in the linear system.

2) Another implementation detail is that when transferring velocity from grid to particles, some of the grid nodes do not have a defined velocity if those grid nodes are only on air cells or solid boundaries. To avoid using undefined velocities for grid-to-particle interpolation, we extrapolate the grid velocity field by one voxel layer. This is implemented in the function `extrapolate_velocity_field` in `fluid_flip_solver.py`. Please also refer to the comments in `FluidFlipSolver.transfer_grid_velocity_to_particles` for the use of the function

Chapter 2

Assignment Requirements

This assignment is to implement the FLIP method in `nemo/solvers/fluid_flip_solver.py`, and in particular its `step` method. I described in the comments the over all structure in the `step` method for you. I also left a few helper functions that I used in my implementation as a reference, but feel free to implement any helper functions you feel needed.

Optional requirement for make-up credits. Some students have requested to do an additional assignment to make up earlier missing PAs. This PA is a chance for that! As I mentioned in the lecture, this entire PA is about *2D* fluid simulation. You can choose to implement a *3D* FLIP fluid simulation method—this requires you to also extend the data structure in `nemo/sim/state.py` and `nemo/sim/model.py` to support 3D fluid simulation. If you choose to do so, you don't need to submit both a 2D and 3D version of the assignment (only 3D version is ok), and your submission will be worth 200 points instead of 100 points in total. In other words, this PA (with 3D FLIP implementation) will be weighted twice as much as a regular PA.

Bibliography

- [1] Robert Bridson. Fluid simulation for computer graphics. In *CRC Press*, 2015.
- [2] Jos Stam. Stable fluids. In *Proceedings of the 26th annual conference on Computer graphics and interactive techniques, ACM*, 1999.
- [3] Yongning Zhu and Robert Bridson. Animating sand as a fluid. *ACM Transactions on Graphics (TOG)*, 24(3):965–972, 2005.